

the `List` and `List Item` components; we would have to pass the `onItemClicked` prop down to where it's utilized. The complexity is amplified if we have multiple props to relay.

The first step is to define a context with an appropriate type:

```
import { createContext } from "react";
import { Item } from "../types";

type SearchableListContextType = {
  onSearch: (keyword: string) => void;
  onItemClicked: (item: Item) => void;
};

const noop = () => {};

const SearchableListContext =
  createContext<SearchableListContextType>({
    onSearch: noop,
    onItemClicked: noop,
  });

export { SearchableListContext };
```

This code defines a React context called `SearchableListContext`. It specifies the types of the `onSearch` and `onItemClicked` functions that the context will hold. It also initializes these functions with a no-operation (`noop`) function by default.

Now, we can use the context as a wrapper in the searchable list to provide the context to all its child components:

```
const SearchableList = ({
  items,
  onSearch,
  onItemClicked,
}: SearchableListProps) => {
  const [filteredItems, setFilteredItems] = useState<Item[]>(items);

  const handleSearch = (keyword: string) => {
    setFilteredItems(items.filter((item) => item.name.
      includes(keyword)));
  };

  return (
    <SearchableListContext.Provider value={{ onSearch, onItemClicked
    }}>
```

```

    <SearchInput onSearch={handleSearch} />
    <List items={filteredItems} />
  </SearchableListContext.Provider>
);
};

```

The `SearchableList` component wraps its children, `SearchInput` and `List`, inside `SearchableListContext.Provider`. This allows these child components to access the `onSearch` and `onItemClicked` functions from the context without passing them down explicitly as props. The `handleSearch` function filters items based on the search keyword.

This means our `List` component can be reverted to the simple version before we introduce `onItemClicked`:

```

const List = ({ items }: { items: Item[] }) => {
  return (
    <section data-testid="searchable-list">
      <ul>
        {items.map((item) => (
          <ListItem item={item} />
        ))}
      </ul>
      <footer>Total items: {items.length}</footer>
    </section>
  );
};

```

For `ListItem`, we can directly access `onItemClicked` from the context:

```

const ListItem = ({ item }: { item: Item }) => {
  const { onItemClicked } = useContext(SearchableListContext);

  return (
    <li onClick={() => onItemClicked(item)}>
      <h2>{item.name}</h2>
      <p>{item.description}</p>
    </li>
  );
};

```

The `ListItem` component now uses `useContext` to access the `onItemClicked` function from `SearchableListContext`. When a list item is clicked, `onItemClicked` is called with the clicked item as an argument.

Likewise, for the `SearchInput` component, there's no need to pass down extra props from `SearchableList`. Instead, we can directly access what we need from the context:

```
const SearchInput = ({ onSearch }: { onSearch: (keyword: string) =>
void }) => {
  const { onSearch: providedOnSearch } =
    useContext(SearchableListContext);

  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    onSearch(e.target.value);
    providedOnSearch(e.target.value);
  };

  return <input type="text" onChange={handleChange} />;
};
```

The `SearchInput` component uses the `useContext` Hook to access the `onSearch` function from `SearchableListContext`. When the input changes, it calls both the local `onSearch` function and the one from the context, effectively merging external and internal behaviors.

As demonstrated, the Context API results in a much cleaner structure. It allows you to make additional structural adjustments to sub-components without the concern of continually passing props down the hierarchy. This simplifies the component interface, making it easier to read and understand.

Summary

In this chapter, we delved into some of the most pressing challenges that are often encountered in React development, such as the leakage of business logic, the complexities associated with prop drilling, and the difficulties of managing shared state. To counter these issues, we introduced robust solutions such as the ACL and the Context API. These strategies aim to streamline your code, making it both more maintainable and effective for long-term projects.

Next up, we'll dive into common React design patterns that you can use to further refine your coding skills. Stay tuned.

Applying Design Principles in React

Design principles are like the cardinal rules that guide software development, ensuring that code remains maintainable, scalable, and readable over time. In the ever-changing landscape of technology, adhering to these principles can be the difference between a project's long-term success and its descent into "code hell," where changes become increasingly arduous and bugs frequent.

For React applications, the importance of design principles escalates due to the library's declarative nature and component-based architecture. React empowers developers to build complex UIs from small, isolated pieces of code known as components. While this modular approach is one of React's strongest features, it can also lead to a messy and unmanageable code base if design principles are ignored.

In a typical React project, components often share state and behavior, get nested within each other, and are reused across different parts of an application. Without following design principles, you might find yourself entangled in a web of dependencies, making it difficult to change or even understand the code. For instance, neglecting the **Single Responsibility Principle (SRP)** could result in components that are difficult to test and refactor, while ignoring the **Interface Segregation Principle (ISP)** could make your components less reusable and more coupled to specific use cases.

Furthermore, as React continues to evolve, with new features such as Hooks and concurrent mode, having a design principle-centered approach ensures that you can adapt to these changes without significant rewrites. This allows you to focus on building features, fixing bugs, and delivering value, instead of grappling with technical debt.

Adhering to design principles in React development is not just a best practice but a necessity. It serves as a proactive measure to counteract complexity, making your React code easier to read, test, and maintain.

In this chapter, we commence by revisiting the SRP, a core concept that often serves as the bedrock of clean, maintainable code. From the humble beginnings of a simple string-transforming function, we'll explore how this principle scales up to the complexities of render props, enriching the structure and readability of your React components.

Transitioning from there, we introduce the **Dependency Inversion Principle (DIP)**, a transformative approach to component design. This section emphasizes that focusing on the interface – not the nitty-gritty details of implementation – is the pathway to reusable and easily understandable components.

Concluding the chapter, we delve into **Command and Query Responsibility Segregation (CQRS)**, a pattern that gains importance as your React applications grow in size and complexity. Through a discussion on CQRS, you'll discover strategies to separate your application's command and query responsibilities, thereby making it more manageable and scalable.

This chapter aims to equip you with a holistic understanding of key design principles, which in turn will lay a strong foundation for the rest of your journey in mastering React.

In this chapter, we will cover the following topics:

- Revisiting the Single Responsibility Principle
- Embracing the Dependency Inversion Principle
- Understanding Command and Query Responsibility Segregation in React

Technical requirements

A GitHub repository has been created to host all the code we discuss in the book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch9>.

Revisiting the Single Responsibility Principle

In *Chapter 4*, we delved into the SRP within the context of designing React components. Yet this principle is more universal, acting as the bedrock for various other programming tenets. To bring this idea to life, let's work through some hands-on examples.

Identifying the core responsibility of a component is key to adhering to the SRP. Once you isolate what the component is fundamentally meant to do, it becomes easier to refactor and abstract out auxiliary functionalities.

The SRP, being a high-level guideline, is advantageous when applied directly at the code level. There are numerous ways to implement this principle, but recognizing when to apply it is crucial, especially as complexity increases.

The most common two techniques we'll use are *render props* and *composition*. Render props refer to a technique in React for sharing code between components using a prop whose value is a function. A component with a render prop takes a function that returns a React element and calls it instead of implementing its own render logic. On the other hand, composition in React is a development pattern where you build components as small, reusable pieces and then compose them together to create more complex UIs.

In the following section, we'll explore two specific examples, demonstrating how we use render props and composition respectively to adhere to this principle in practice.

Exploring the render props pattern

Let's start simple with a basic function component called `Title`:

```
const Title = () => <div>Title | This is a title</div>
```

As it stands, this component merely outputs a static string. To give it the ability to render different titles, we introduce a `title` prop:

```
const Title = ({ title }: { title: string }) => <div>Title |  
{title}</div>;
```

With this change, the component becomes more versatile, appending a fixed prefix, `Title |`, to any title we pass in. But what if we want to further manipulate the title, perhaps to capitalize it?

By utilizing a higher-order function – the `transformer` parameter in the following code snippet – we can modify our `Title` component as follows:

```
const Title = ({  
  title,  
  transformer,  
}: {  
  title: string;  
  transformer: (s: string) => string;  
}) => <div>Title | {transformer(title)}</div>;
```

Note

In many programming languages (including JavaScript), a **higher-order function** is one that either takes another function as an argument, returns a function, or both. These functions are foundational in functional programming, enabling you to write modular and reusable code. They are commonly used for operations such as `map`, `filter`, and `reduce` on arrays, function composition, currying, and event handling. Higher-order functions simplify code structure, improve maintainability, and allow for more advanced programming techniques.

Great – our title is now fully customizable. But let's stretch this even further. What if we want the title to be inside an `h3` tag rather than a simple `div` tag? React has got us covered – we can pass a function that returns JSX elements:

```
const Title = ({  
  title,  
  render,
```

```
  }: {
    title: string;
    render: (s: string) => React.ReactNode;
  }) => <div>{render(title)}</div>;
```

Notice the use of the render prop – we call it as a function and pass in `title`.

To use the render prop, we can pass an anonymous function (inside the curly braces) into it, as in the following code:

```
<Title
  title="This is a title"
  render={(s: string) => {
    const formatted = s.toUpperCase();
    return <h3>{formatted}</h3>;
  }}
/>
```

In React, this higher-order function doesn't necessarily have to be named `render`. We could just as easily use the `children` prop for a more intuitive design, like so:

```
const Title = ({
  title,
  children,
}): {
  title: string;
  children: (s: string) => React.ReactNode;
}) => <div>{children(title)}</div>;
```

This allows us to invoke `children` as if it were a regular function:

```
<Title title="This is a title">
  {(s: string) => {
    const formatted = s.toUpperCase();
    return <h3>{formatted}</h3>;
  }}
</Title>
```

The `Title` component receives a `title` prop and a child function (the latter of which is called a **render prop**). The child function takes a string, `s`, converts it to uppercase, and renders it within an `h3` tag. The `Title` component calls this child function with the provided `title` prop for custom rendering.

In React, the render prop pattern involves passing a function as a prop to a component. This function returns JSX that the component will render as part of its output. The pattern allows for more flexible and reusable components by giving the parent component control over a part of the child component's rendering logic. It's particularly useful for sharing behavior across multiple components.

Take note of the overarching pattern at play here: abstraction. Initially, we might think of `h2` or `h3` as specific instances of headings. However, upon zooming out a bit, we start to understand that they're part of a broader abstraction: a React component or, more technically, `ReactNode`.

This realization allows us to see the utility of using render props or children as higher-order functions. They're not just features; they represent the level of abstraction we've achieved. Now, instead of being limited to a specific HTML tag such as `h3`, we can pass any JSX element as an argument, from headings to fully styled components.

With our newly crafted generic component that uses a render prop, we've essentially created a reusable framework. The beauty lies in the fact that we only need to write this general-purpose code once.

Render props and composition are excellent techniques for this. They allow you to extend or customize the behavior of a component without altering its core logic. This keeps your components clean, modular, and easy to test, as each component does one thing and does it well. We have already seen how render props work in the evolution of `Title`, so let's now have a look at composition.

Using composition to apply the SRP

Composition is a term we have used in many places throughout the book, and at its core is the SRP. If each part of the system can do its job well, it's then possible to compose them together. Let's inspect a concrete example of this.

Assume we have an `Avatar` component in a design system with a handy feature: if a user passes in a `name` prop to the component, then when the mouse hovers over the avatar, a tooltip will show up at the bottom of the avatar with the name as its content:

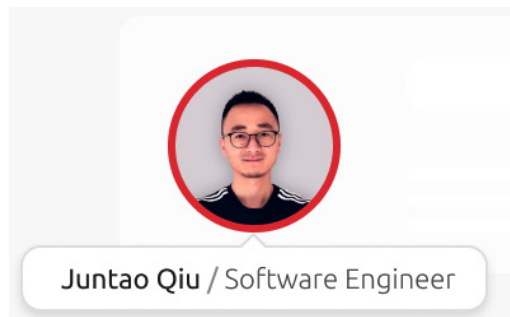


Figure 9.1: Avatar component with Tooltip

Internally, `Avatar` utilizes another component, `Tooltip`, to make it happen:

```
import Tooltip from "@xui/tooltip";

type AvatarProps = {
  name?: string;
  url: string;
};

const Avatar = ({ name, url }: AvatarProps) => {
  if (name) {
    return (
      <Tooltip content={name}>
        <div className="rounded">
          <img src={url} alt={name} />
        </div>
      </Tooltip>
    );
  }
  return (
    <div className="rounded">
      <img src={url} alt="" />
    </div>
  );
};
```

The `Avatar` component takes two optional props, `name` and `url`, and displays an image using the URL provided. If the `name` prop is also provided, it wraps the image in a `Tooltip` component that shows the name when hovered over. The `div` tag is styled with a `rounded` class, which will present the avatar in a circle.

The original code for the `Avatar` component tightly coupled it to a `Tooltip` feature. As users demanded more customization options for the tooltip, maintaining this coupling became challenging. Adding more props to handle tooltip customization can bloat the `Avatar` component and create a ripple effect: any change in `Tooltip` may necessitate changes in `Avatar`, making it hard to manage.

Instead of forcing `Tooltip` into `Avatar`, we can simplify `Avatar` to focus solely on its primary function—displaying an image. This stripped-down version excludes the tooltip, reducing its bundle size and making it more maintainable. Here's how the simplified `Avatar` component looks:

```
const Avatar = ({ name = "", url }: AvatarProps) => (
  <div className="rounded">
    <img src={url} alt={name} title={name} />
  </div>
);
```

By doing so, we make the `Avatar` and `Tooltip` components composable, meaning they can work independently of each other. The consumer can then choose to wrap `Avatar` with `Tooltip` if desired, as shown in the following code snippet:

```
import Avatar from "@xui/avatar";
import Tooltip from "@xui/tooltip";

const MyAvatar = (props) => (
  <Tooltip
    content="Juntao Qiu"
  >
    <Avatar
      name="Juntao Qiu"
      url="https://avatars.githubusercontent.com/u/122324"
    />
  </Tooltip>
);
```

The code imports `Avatar` and `Tooltip` components from the `"@xui"` library. It then defines a `MyAvatar` component that displays an avatar for `"Juntao Qiu"` (if there isn't a name needed here, we don't use the `Tooltip` component). When you hover over the avatar, a tooltip appears on top with the name **Juntao Qiu** in a design customized with a white font color on a blue background.

The benefit of this approach is twofold:

- The `Avatar` component remains lean, reducing its bundle size
- The consumer has the freedom to customize `Tooltip` or even use different tooltip libraries without affecting `Avatar`

In short, the separation makes the code more modular, and users only have to “pay” in terms of code and complexity for the features they actually use.

In both the render props and composition examples, we underlined the essence of the SRP in modern web development. The SRP advocates for building components that do one thing and do it well, making them more maintainable, reusable, and flexible.

Next, let's pivot to discussing the DIP, another crucial perspective that complements these design principles.

Embracing the Dependency Inversion Principle

The **DIP** is one of the five principles that make up SOLID, a set of guidelines aimed at helping developers create more maintainable, flexible, and scalable software. Specifically, the DIP encourages developers to depend on abstractions, not on concrete implementations.

Note

The five principles of SOLID are the Single Responsibility Principle, Open-Closed Principle, Liskov Substitution Principle, Interface Segregation Principle, and Dependency Inversion Principle.

The DIP addresses several challenges that developers face when building and maintaining large systems. One such problem is the rigidity that comes from tightly coupled modules. When high-level modules are dependent on low-level modules, even small changes to the low-level code can have a broad impact, necessitating changes across the system.

Understanding how the DIP works

In terms of high-level modules and low-level modules, let's think of a notification feature in a system. Here, we want to send out a notification in a form that the user prefers, either an email, an SMS message, or both:

```
class EmailNotification {
  send(message: string, type: string) {
    console.log(`Sending email with message: ${message}, type:
${type}`);
  }
}

class Application {
  private emailNotification: EmailNotification;

  constructor(emailNotification: EmailNotification) {
    this.emailNotification = emailNotification;
  }

  process() {
    // perform some actions to response user interaction
    this.emailNotification.send("Some events happened", "info");
  }
}

const app = new Application(new EmailNotification());
app.process();
```

In the code, the `EmailNotification` class has a method called `send` that takes `message` and `type` as parameters. It then prints out a log to indicate that an email with this message and type is being sent. The `Application` class, on the other hand, has a `process` method that simulates some kind of user interaction. Inside this method, `Application` uses an instance of `EmailNotification` to send an email whenever `process` is invoked.

One important thing to note here is that `Application` is tightly coupled to `EmailNotification`. This means that if you wanted to change how notifications are sent, perhaps by using SMS instead of email, you'd have to modify the `Application` class directly, thereby violating the SRP and making the system less flexible.

So, to resolve the problem, we can introduce a `Notification` interface and let `EmailNotification` implement the interface. That means we could have multiple implementations of the interface. Plus, instead of `Application` depending on the `EmailNotification` class, `Application` depends on the `Notification` interface. Because we rely on the interface, from `Application`'s view, it doesn't matter which concrete implementation is passed in, as long as it implements the `Notification` interface – that means we could easily swap it to an `SMSNotification` class if we like. Here's what the code for all of this will look like:

```
interface Notification {
    send(message: string, type: string): void;
}

class EmailNotification implements Notification {
    send(message: string, type: string) {
        console.log(`Sending email with message: ${message}, type:
            ${type}`);
    }
}

class Application {
    private notifier: Notification;

    constructor(notifier: Notification) {
        this.notifier = notifier;
    }

    process() {
        // perform some actions to response user interaction
        this.notifier.send("Some event happened", "info");
    }
}
```

The code defines a `Notification` interface with a `send` method, which is then implemented by the `EmailNotification` class. The `Application` class is now constructed with any object that adheres to the `Notification` interface. Within its `process` method, `Application` uses this object to send a notification. This setup decouples the `Application` class from the specific notification mechanism, making it more flexible and easier to change or extend.

For instance, if we decide to replace `EmailNotification` with `SMSNotification`, the `Application` class won't need any modifications; we would simply provide a different instance that implements the `Notification` interface:

```
const app = new Application(new EmailNotification());
app.process();

// or
const app = new Application(new SMSNotification());
app.process();
```

All right – that's briefly about how the DIP works. Let's look at another example to find out how to apply the same principle inside a React application.

Applying the DIP in an analytics button

Now, imagine you have a generic button component that's used across various parts of your application. You want to send analytics events when the button is clicked, but how exactly those events are sent should be abstracted away from the button component itself.

The problem is that generic buttons are widely used in many products already, and not all of them need the analytics functionality. So, if you simply change the `onClick` handler in the shared `Button` component, it will annoy many innocent users.

Let's have a look at the current `Button` implementation:

```
const Button = ({ onClick: provided, name, ...rest }: ButtonProps) => {
  {
    const onClick = (e) => {
      // emit an event to the analytic server
      return provided(e);
    };

    return <button onClick={onClick} {...rest} />;
  }
};
```

Instead, we could define a new component that wraps the original button around and hijacks the click handler for the analysis:

```
import { Button } from "@xui/button";

const FancyButton = ({
  onClick: originalOnClick,
  ...rest
}: FancyButtonProps) => {
  const onClick = (e) => {
```

```
//emit an event to the analytic server
console.log('sending analytics event to a remote server');
return originalOnClick(e);
};

return <Button onClick={onClick} {...rest} />;
};
```

The new code defines a `FancyButton` component that wraps around a basic `Button` component. When clicked, `FancyButton` first sends an analytics event to a remote server and then proceeds to execute the original `onClick` function passed to it. All other props are passed down directly to the underlying `Button` component.

The issue here is that many instances where the `Button` component is used might contain similar analytics code, leading to repetitive logic across the code base. This redundancy is undesirable, as any changes to the analytics logic would require updates in multiple locations, increasing the risk of errors.

So, let's consider the DIP. We will make some changes in the original `Button` component, but instead of sending analytics events directly, we'll first extract an interface and make the button rely on the interface (keep in mind there could be multiple implementations of that interface).

Just as with the previous `Notification` example, `EmailNotification` is one of the notification channels that send email. In this button example, one of the implementations sends an event, whereas for products that don't use analytics at all, they just pass in an empty implementation.

To make the change, we'll need to define a new interface type, and we need a context for the implementation of the interface to live in:

```
import { createContext } from "react";

export interface InteractionMeasurement {
  measure(name: string | undefined, timestamp?: number): void;
}

export default createContext<InteractionMeasurement | null>(null);
```

This code creates a React context named `InteractionMeasurement` with an interface that specifies a `measure` method. This method takes a name (either a string or undefined) and an optional timestamp, while the context is initialized as `null`.

Inside the `Button` component, we can use `useContext` to access the context we defined:

```
import InteractionContext, {
  InteractionMeasurement
} from "../InteractionContext";

const Button = ({ name, onClick: providedOnClick, children }:
```

```

ButtonType) => {
  const interactionContext = useContext<InteractionMeasurement |
    null>(
    InteractionContext
  );

  const handleClick = useCallback(
    (e) => {
      interactionContext &&
        interactionContext.measure(name, e.timeStamp);
      providedOnClick(e);
    },
    [providedOnClick, interactionContext, name]
  );

  return <button onClick={handleClick}>{children}</button>;
};

```

The code defines a `Button` component that uses `InteractionContext` to track clicks. When the button is clicked, it calls the `measure` method from the context, passing in the button's name and the click event's timestamp. Then, it proceeds to execute any additional `onClick` logic provided. This way, click tracking is abstracted away into the context, making the `Button` component more reusable and maintainable.

If `interactionContext` is `null`, the `measure` function won't be called, and the component will proceed to execute only the `providedOnClick` function passed in as a prop. This allows for optional analytics tracking based on the availability of `InteractionContext`.

That would perfectly resolve the problem we have – if a product wants to enable the analytics, they can use `Button` within a context that contains an `InteractionMeasurement` implementation.

Having said that, let's say we have a `FormApp` application that uses `Button` inside an `InteractionContext` instance:

```

import InteractionContext from "../InteractionContext";
import { Button } from "@xui/button";

const FormApp = () => {
  const context = {
    measure: (e, t) => {
      //send event and timestamp to remote
      console.log(`sending to remote server  ${e}: ${t}`);
    },
  };

  const onClick = () => {

```

```
    console.log("submit");
  };

  return (
    <InteractionContext.Provider value={context}>
      <form>
        <Button name="submit-button" onClick={onClick}>
          Submit
        </Button>
      </form>
    </InteractionContext.Provider>
  );
};
```

The `FormApp` component defines its own analytics logic in the `measure` function inside the `context` object. It then passes this function to child components through `InteractionContext.Provider`. When a button inside the form is clicked, not only will the button's specific `onClick` logic be executed, but the `measure` function will also send event and timestamp data to a remote server for analytics. This setup allows for context-based analytics without tying the `Button` component to a specific implementation.

For users who don't want the analytics functionality, they can just use the `Button` component as usual:

```
import { Button } from "@xui/button";

const App = () => {
  const onClick = () => {
    console.log("checkout");
  };

  return (
    <Button name="checkout-button" onClick={onClick}>
      Checkout
    </Button>
  );
};
```

This methodology offers exceptional flexibility and dynamism, making it invaluable for designing common components. It enhances both code reusability and system maintainability while also reducing the overall bundle size.

Be aware that adding an extra `context` object in such scenarios may initially seem excessive. However, in large code bases where different teams work on distinct parts, this approach becomes more relevant. For instance, a product team focused on analysis might have different objectives compared to a design

system team, whose aim is to develop generic and atomic components. The design system team may not be concerned with analytical aspects. Consequently, directly modifying the `Button` component in this environment can be impractical or challenging.

Having that said, I would like to introduce another design principle I constantly use in my code; you can think of it as a special form of the SRP at its core. This principle is CQRS.

Understanding Command and Query Responsibility Segregation in React

The **Command and Query Responsibility Segregation (CQRS)** principle (also known as the **Separation of Command and Query Principle**) is a software design principle that suggests that methods or functions should either be commands that modify the system's state or queries that return information about the system's state, but not both.

Commands (or **modifiers**) are methods that perform an action or change the state of an object without returning a value. **Queries**, on the other hand, are methods to read an object's state without any changes. Separating commands and queries can help reduce coupling between components, making testing, maintaining, and modifying code easier. It also makes it easier to reason about the behavior of code and can improve the overall design of a system.

Although this pattern is widely used on a large scale, such as in designing the architecture of systems, it works well at the code level as well. I will demonstrate this in a `ShoppingCart` component:

```
type Item = {
  id: string;
  name: string;
  price: number;
}

const ShoppingApplication = () => {
  const [cart, setCart] = useState<Item[]>([]);

  const addItemToCart = (item: Item) => {
    setCart([...cart, item]);
  };

  const removeItemFromCart = (id: string) => {
    setCart(cart.filter((item) => item.id !== id));
  };

  const totalPrice = cart.reduce((total, item) => total + item.price,
    0);

  return (
```

```

<div>
  <ProductList addToCart={addItemToCart} />

  <h2>Shopping Cart</h2>
  <ul>
    {cart.map((item) => (
      <li key={item.id}>
        {item.name} - {item.price}
        <button onClick={() => removeItemFromCart(item.
          id)}>Remove</button>
      </li>
    ))}
  </ul>
  <p>Total Price: {totalPrice}</p>
</div>
);
};

```

The `ShoppingApplication` component maintains a shopping cart using the `useState` Hook with an array of items of type `Item`. The `addItemToCart` function adds new items to the cart, and `removeItemFromCart` removes items based on their `id` value. `totalPrice` is calculated as the sum of all item prices in the cart.

The component renders a list of items in the cart, along with their total price. Each item has a **Remove** button that calls `removeItemFromCart` when clicked. A `ProductList` component is also rendered, and it receives `addItemToCart` as a prop for adding products to the cart.

The previous code appears to be okay at first glance, but it contains some subtle issues. One problem is that when multiple identical products are added to the cart, the keys will overlap, triggering React's warning about unique keys. Additionally, if you click the **Remove** button in this situation, it will delete all instances of that product from the cart, which is far from ideal and leads to a poor user experience.

To fix these issues, we need to introduce a new `uniqKey` field to the `Item` type. We also need to generate a unique key before an item is inserted into the `cart` array. With that unique ID, we are finally able to remove items by `uniqKey` instead of by `id`. This is what the code will look like:

```

const addItemToCart = (item: Item) => {
  setCart([...cart, { ...item, uniqKey: `${item.id}-${Date.now()}` }]);
};

const removeItemFromCart = (key: string) => {
  setCart(cart.filter((item) => item.uniqKey !== key));
};

```

We also need to update how the cart is rendered in JSX:

```
<h2>Shopping Cart</h2>
<ul>
  {cart.map((item) => (
    <li key={item.uniqKey}>
      {item.name} - {item.price}
      <button onClick={() => removeItemFromCart(item.uniqKey)}>
        Remove
      </button>
    </li>
  ))}
</ul>
```

While the code is technically sound and sufficiently straightforward for its current scope, as we expand the `ShoppingApplication` component with more states and calculations, applying the CQRS principle could provide a structured way to keep everything organized.

We'll use the React Context API and the `useReducer` Hook to implement CQRS for the `ShoppingApplication` component. Let's take a look at them now.

Introducing `useReducer`

The `useReducer` Hook in React is used for state management in functional components; it is particularly useful when the next state depends on the previous one or when you have complex state logic. The `useReducer` Hook takes two arguments: a reducer function and an initial state, and it returns the current state and a `dispatch` method to trigger updates.

For the first parameter, a reducer function receives the current state and an `action` object, which contains information on how to update the state. The function should return the new state based on the action type and payload. The second parameter is the initial state you want to pass in and will be used as the default value when invoked.

Let's define a reducer function for our `ShoppingApplication` component:

```
type ShoppingCartState = {
  items: Item[];
  totalPrice: number;
};

type ActionType = {
  type: string;
  payload: Item;
};

const shoppingCartReducer = (
```

```
state: ShoppingCartState = initState,
action: ActionType
) => {
  switch (action.type) {
    case "ADD_ITEM": {
      const item = {
        ...action.payload,
        uniqKey: `${action.payload.id}-${Date.now()}`,
      };
      return { ...state, items: [...state.items, item] };
    }

    case "REMOVE_ITEM":
      const newItems = state.items.filter(
        (item) => item.uniqKey !== action.payload.uniqKey
      );
      return { ...state, items: newItems };
    default:
      return state;
  }
};
```

So, `shoppingCartReducer` is a function that takes two arguments – the current state and an action:

- The state is of type `ShoppingCartState`, which includes an array of items and `totalPrice`
- The action is of type `ActionType`, which includes a string type to identify the action and a payload containing an `Item` object

Inside the reducer function, a `switch` statement is used to determine which action is being dispatched. The `"ADD_ITEM"` case adds a new item to the state's `items` array. This item is given a unique key, `uniqKey`, to differentiate it from identical items. The `"REMOVE_ITEM"` case removes an item from the `items` array based on this unique key.

By using this structure, the reducer function provides a predictable way to manage the shopping cart's state in response to different actions. Note there is nothing fancy here in this reducer function; it's just a regular JavaScript function. To see how it works, we could test the reducer function with the following code:

```
const item = {
  id: "p1",
  name: "iPad",
  price: 666,
};

let x = shoppingCartReducer(initState, {
```

```
    type: "ADD_ITEM",  
    payload: item,  
  });  
  
  console.log(x);
```

And we would get something like this (obviously, your `uniqKey` value would be different from mine as it's generated by the time an item is added):

```
{  
  "items": [  
    {  
      "id": "p1",  
      "name": "iPad",  
      "price": 666,  
      "uniqKey": "p1-1696059737801"  
    }  
  ],  
  "totalPrice": 0  
}
```

All right – that should give a taste of what a reducer function is and how it works with any given input. Now, let's see how we can connect it with our application.

Using a reducer function in a context

Let's see how we can use a reducer function to implement CQRS to simplify our shopping cart example. Firstly, we'll need a context to manage the cart state, and also expose query functions for components to use:

```
import React, { createContext, useContext, useReducer } from "react";  
import { Item } from "../type";  
  
type ShoppingCartContextType = {  
  items: Item[];  
  addItem: (item: Item) => void;  
  removeItem: (item: Item) => void;  
};  
  
const ShoppingCartContext = createContext<ShoppingCartContextType |  
null>(null);  
  
export const ShoppingCartProvider = ({  
  children,
```

```
}: {
  children: React.ReactNode;
}) => {
  const [state, dispatch] = useReducer(shoppingCartReducer, {
    items: [],
    totalPrice: 0,
  });

  const addItem = (item: Item) => {
    dispatch({type: ADD_ITEM, payload: item});
  };

  const removeItem = (item: Item) => {
    dispatch({type: REMOVE_ITEM, payload: item});
  };

  return (
    <ShoppingCartContext.Provider value={{items: state.items, addItem,
      removeItem}}>
      {children}
    </ShoppingCartContext.Provider>
  );
};
```

This code creates a React context for managing a shopping cart. Inside `ShoppingCartProvider`, it uses the `useReducer` Hook to handle cart actions. Two functions, `addItem` and `removeItem`, dispatch actions to modify the cart. The `Provider` component makes the cart state and these functions available to its child components via `ShoppingCartContext`. This allows any nested components to interact with the shopping cart.

Note that `addItem` and `removeItem` are two command functions in the CQRS principle that only change the state without returning any data. If we want to get the data, we can define a query function, like so:

```
export const useTotalPrice = () => {
  const context = useContext<ShoppingCartContextType>(
    ShoppingCartContext
  );

  const {items} = context;

  return items.reduce((acc, item) => acc + item.price, 0);
};
```

Here, we define a custom Hook called `useTotalPrice` that calculates the total price of items in a shopping cart. It uses the React `useContext` Hook to access the shopping cart data from `ShoppingCartContext`. It then uses the `reduce` method to sum up the prices of all items in the cart, starting with an initial value of 0.

For the `ShoppingApplication` component, we can simply wrap `ProductList` and `ShoppingCart` inside the `ShoppingCartContext` instance we just created:

```
const ShoppingApplication = () => {
  const context = useContext(ShoppingCartContext);
  const { items, addItem, removeItem } = context;
  const totalPrice = useTotalPrice();

  return (
    <div>
      <ProductList addToCart={addItem} />

      <h2>Shopping Cart</h2>
      <ul>
        {items.map((item) => (
          <li key={item.uniqKey}>
            {item.name} - {item.price}
            <button onClick={() => removeItem(item)}>Remove</button>
          </li>
        ))}
      </ul>
      <p>Total Price: {totalPrice}</p>
    </div>
  );
};
```

The `ShoppingApplication` component serves as the main interface for the shopping application. It uses React's `useContext` Hook to access the shopping cart context, which provides a list of items in the cart (`items`), a function to add items (`addItem`), and a function to remove items (`removeItem`). The component also uses a `useTotalPrice` custom Hook to calculate the total price of items in the cart.

And in the outmost `App` component, we can encapsulate the `ShoppingApplication` component:

```
<ShoppingCartProvider>
  <ShoppingApplication />
</ShoppingCartProvider>
```

So, CQRS is a design pattern that separates the modification and query aspects of a system to enhance scalability, maintainability, and simplicity. We demonstrated this principle by implementing a shopping cart feature – commands to modify the cart’s state, such as adding or removing items, were segregated from the queries, which included fetching a list of items and calculating the total price; this separation was made clear through the use of React’s Context API and custom Hooks, which isolated each responsibility effectively. This not only improves code readability but also makes it easier to manage and scale the application in the future.

Summary

In this chapter, we’ve unpacked three crucial design principles: the SRP for focused, easy-to-understand components, the DIP for modular, testable code, and CQRS for a distinct separation between commands and queries, enhancing maintainability. These principles offer a robust foundation for building scalable and high-quality software.

In the next chapter, we’ll dive deeper into composition principles to further refine our approach to React application design.

10

Diving Deep into Composition Patterns

The journey of building scalable and maintainable user interfaces is riddled with challenges. One primary challenge faced by developers is ensuring that components remain modular, reusable, and easy to understand as a code base grows. The more intertwined and tightly coupled our components become, the harder it is to maintain, test, or even onboard new team members.

Composition has emerged as a powerful technique to address this challenge, enabling developers to build more organized, scalable, and cleaner code bases. Instead of creating large, monolithic components that carry out numerous tasks, we break them down into smaller, more manageable pieces that can be combined in versatile ways. This offers us a clear path to streamline logic, enhance reusability, and maintain a clear separation of concerns.

This chapter is dedicated to understanding and mastering composition in React. We'll delve into foundational techniques such as higher-order functions before transitioning into higher-order components and Hooks. You'll learn how these tools seamlessly align with the principles of composition, allowing you to build more robust applications with React. Our journey will culminate with a deep dive into headless components, a paradigm that encapsulates logic without dictating the UI, offering unparalleled flexibility.

By the chapter's end, you'll appreciate the benefits of employing composition techniques. You'll be equipped to create UIs that are not just scalable and maintainable but also a pleasure to work with. Let's embark on this enlightening exploration of composition in React.

In this chapter, we will cover the following topics:

- Understanding composition through higher-order components
- Diving deep into custom Hooks
- Developing a drop-down list component
- Exploring a headless component pattern

Technical requirements

A GitHub repository has been created to host all the code we discuss in the book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch10>.

Understanding composition through higher-order components

Composition might be the most important technique in software design overall, and like many other fundamental design principles, it applies on many different levels. In this section, we'll review how we can use higher-order functions and their variation in the React world – higher-order components – to implement composition.

Reviewing higher-order functions

We discussed some examples of higher-order functions in *Chapter 9*, but it's such an important concept that I would like to review it a bit more here. A **higher-order function (HOF)** is a function that either takes another function as its argument, returns a function, or both. The ability to accept a function as a parameter has a lot of advantages, especially when it comes to composition.

Consider the following example:

```
const report = (content: string) => {
  const header = "=== Header ===";
  const footer = "=== Footer ===";

  return [header, content, footer].join("\n");
};
```

Here, the `report` function generates a formatted report containing a header, the provided content, and a footer. For instance, given the input `hello world`, the output would be as follows:

```
=== Header ===
hello world
=== Footer ===
```

Now, imagine a scenario where some users wish to print the content in uppercase. While we could achieve this with `content.toUpperCase()`, other users might prefer the content as-is. Introducing conditions within our `report` function is one approach to pleasing both sets of users. Drawing inspiration from our previous discussion about the title example in *Chapter 9*, we can allow a `transformer` function to be passed.

This enables clients to format the string as they desire, like so:

```
const report = (content: string, transformer: (s: string) => string)
=> {
  const header = "=== Header ===";
  const footer = "=== Footer ===";

  return [header, transformer(content), footer].join("\n");
};
```

For flexibility, we can provide a default transformer, ensuring that those who don't wish to customize the format can use the function without changes:

```
const report = (
  content: string,
  transformer: (s: string) => string = (s) => s
) => {
  const header = "=== Header ===";
  const footer = "=== Footer ===";

  return [header, transformer(content), footer].join("\n");
};
```

The report function generates a string with a defined header and footer, and the main content in between. It accepts a content string and an optional transformer function. If the transformer is provided, it modifies the content; otherwise, the content remains unchanged. The result is a formatted report with the modified or original content placed between the header and footer. That's essentially how powerful HOFs can be, helping us to write more composable code.

Reflecting upon this, an interesting thought emerges – can we incorporate this composable and functional approach into our React applications? Indeed, we can. The ability to augment components isn't just limited to standard functions. In React, we have **higher-order components (HOCs)**.

Introducing HOCs

An HOC is essentially a function that accepts a component and returns a new, enhanced version of it. The principle behind HOCs is straightforward – they allow you to inject additional functionality into an existing component. This pattern is especially beneficial when you want to reuse certain behaviors across multiple components.

Let's delve into an example:

```
const checkAuthorization = () => {
  // Perform authorization check, e.g., check local storage or send
  a request to a remote server
}
```

```
}

const withAuthorization = (Component: React.FC): React.FC => {
  return (props: any) => {
    const isAuthorized = checkAuthorization();
    return isAuthorized ? <Component {...props} /> : <Login />;
  };
};
```

In this snippet, we define a function, `checkAuthorization`, to handle the authorization check. Then, we create a HOC, `withAuthorization`. This HOC takes a component (`Component`) as its argument and returns a new function. This returned function, when rendered, will either render the original `Component` (if the user is authorized) or a `Login` component (if the user is not authorized).

Now, suppose we have a `ProfileComponent` that we want to secure. We can use `withAuthorization` to create a new, secured version of `ProfileComponent`:

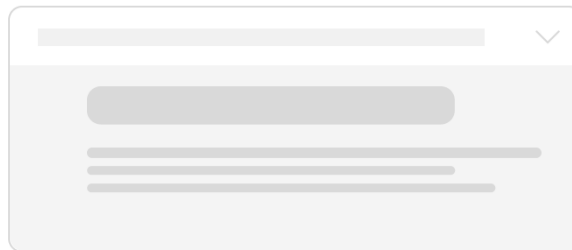
```
const Profile = withAuthorization(ProfileComponent);
```

This means that whenever `Profile` is rendered, it will first check whether a user is authorized. If so, it renders `ProfileComponent`; otherwise, it redirects the user to the `Login` component.

Now that we've seen how HOCs can control access with `withAuthorization`, let's shift our focus to enhancing user interactions. We'll delve into an `ExpandablePanel` component, showcasing how HOCs can also manage interactive UI elements and state transitions.

Implementing an `ExpandablePanel` component

Let's kick things off with a basic `ExpandablePanel` component. This component, as the name suggests, consists of a title and a content area. Initially, the content area is collapsed, but a click on the title expands it to reveal the content.



ExpandablePanel

Figure 10.1: An expandable panel

The code for such a component is straightforward:

```
export type PanelProps = {
  heading: string;
  content: ReactNode;
};

const ExpandablePanel = ({ heading, content }: PanelProps) => {
  const [isOpen, setIsOpen] = useState<boolean>(false);

  return (
    <article>
      <header onClick={() => setIsOpen((isOpen) =>
        !isOpen)}>{heading}</header>
      {isOpen && <section>{content}</section>}
    </article>
  );
};
```

Now, suppose we want to jazz it up a bit, making the panel expand automatically when rendered and then collapse after a few seconds. Here's how we could adjust the code to achieve that:

```
const AutoCloseExpandablePanel = ({ heading, content }: PanelProps) => {
  const [isOpen, setIsOpen] = useState<boolean>(true);

  useEffect(() => {
    const id = setTimeout(() => {
      setIsOpen(false);
    }, 3000);

    return () => {
      clearTimeout(id);
    };
  }, []);

  return (
    <article>
      <header onClick={() => setIsOpen((isOpen) =>
        !isOpen)}>{heading}</header>
      {isOpen && <section>{content}</section>}
    </article>
  );
};
```

In this revised version, we initialize `isOpen` to `true` so that the panel starts as expanded. Then, we utilize `useEffect` to set a timer that collapses the panel after 3,000 milliseconds (3 seconds).

This pattern of auto-collapsing components is quite common in UI development – think of notifications, alerts, or tooltips that disappear after a while. To promote code reusability, let's extract this auto-collapsing logic into a HOC:

```
interface Toggleable {
  isOpen: boolean;
  toggle: () => void;
}

const withAutoClose = <T extends Partial<Toggleable>>(<
  Component: React.FC<T>,
  duration: number = 2000
) => (props: T) => {
  const [show, setShow] = useState<boolean>(true);

  useEffect(() => {
    if (show) {
      const timerId = setTimeout(() => setShow(false), duration);
      return () => clearTimeout(timerId);
    }
  }, [show]);

  return (
    <Component
      {...props}
      isOpen={show}
      toggle={() => setShow((show) => !show)}
    />
  );
};
```

In `withAutoClose`, we define a generic HOC that adds auto-closing functionality to any component. This HOC accepts a duration parameter to customize the auto-close delay, defaulting to 2,000 milliseconds (2 seconds).

To ensure a smooth integration, we can also extend `PanelProps` to include optional `Toggleable` properties:

```
type PanelProps = {
  heading: string;
  content: ReactNode;
} & Partial<Toggleable>;
```

Now, we can refactor `ExpandablePanel` to accept `isOpen` and `toggle` props from `withAutoClose`:

```
const ExpandablePanel = ({
  isOpen,
  toggle,
  heading,
  content,
}: PanelProps) => {
  return (
    <article>
      <header onClick={toggle}>{heading}</header>
      {isOpen && <section>{content}</section>}
    </article>
  );
};
```

With this setup, creating an auto-closing version of `ExpandablePanel` is a breeze:

```
export default withAutoClose(ExpandablePanel, 3000);
```

And guess what? The auto-closing logic we've encapsulated in `withAutoClose` can be reused across various components:

```
const AutoDismissToast = withAutoClose(Toast, 3000);
const TimedTooltip = withAutoClose(Tooltip, 3000);
```

The versatility of HOCs shines when it comes to composition – the ability to apply one HOC to the result of another. This capability aligns well with the principle of function composition in functional programming.

Let's consider another HOC, `withKeyboardToggle`, which augments a panel's behavior to respond to keyboard inputs to toggle the panel's expanded/collapsed state. Here's the code for `withKeyboardToggle`:

```
const noop = () => {};

const withKeyboardToggle =
  <T extends Partial<Toggleable>>(Component: React.FC<T>) =>
  (props: T) => {
    const divRef = useRef<HTMLDivElement>(null);

    const handleKeyDown = (event: KeyboardEvent<HTMLDivElement>) => {
      if (event.key === "Enter" || event.key === " ") {
        event.preventDefault();
        (props.toggle ?? noop)();
      }
    };
  }
```



```
    }

    if (event.key === "Escape" && divRef.current) {
      divRef.current.blur();
    }
  };

  return (
    <div onKeyDown={handleKeyDown} tabIndex={0} ref={divRef}>
      <Component {...props} />
    </div>
  );
};

export default withKeyboardToggle;
```

In the `withKeyboardToggle` HOC, a reference (`divRef`) is created for the wrapping `div` to enable keyboard interactions. The `handleKeyDown` function defines the behavior for the *Enter*, *Space*, and *Escape* keys – the *Enter* or *Space* keys toggle the panel's state, while the *Escape* key removes focus from the panel. These keyboard event handlers allow the wrapped component to respond to keyboard navigation.

Now, let's compose `withKeyboardToggle` and `withAutoClose` together to create a new component, `AccessibleAutoClosePanel`:

```
const AccessibleAutoClosePanel =
  withAutoClose(withKeyboardToggle(ExpandablePanel), 2000);
```

In the `withAutoClose(withKeyboardToggle(ExpandablePanel), 2000);` expression, `withKeyboardToggle` is first applied to `ExpandablePanel`, enhancing it with keyboard toggle capability. The result is then fed into `withAutoClose`, which further enhances the component to auto-close after a 2,000-millisecond delay. This chaining of HOCs results in a new component, `AccessibleAutoClosePanel`, which inherits both the keyboard toggle and auto-close behaviors.

This is a vivid example of how HOCs can be nested and composed to build more complex behavior from simpler, single-responsibility components, which is illustrated further in *Figure 10.2*:

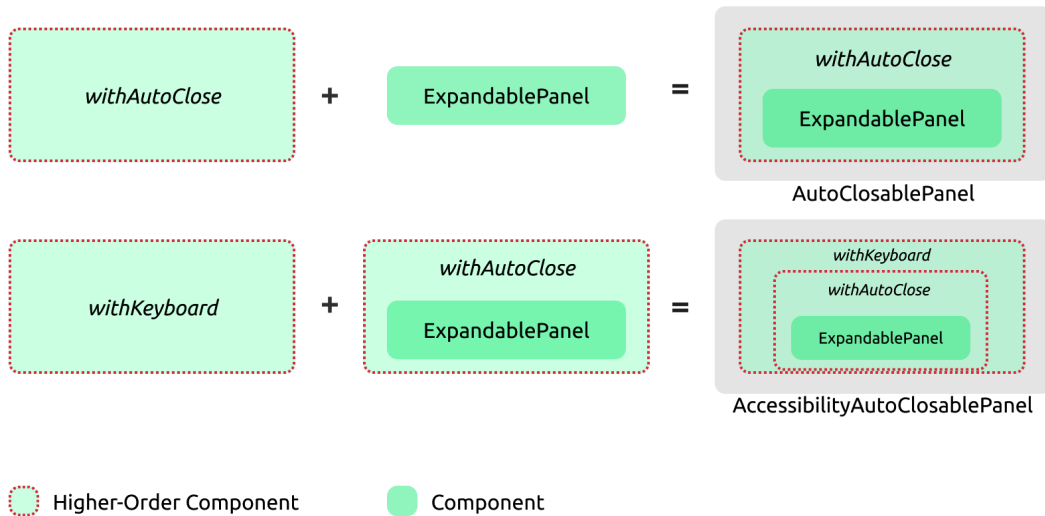


Figure 10.2: A Higher-Order Component

If you have some background in object-oriented programming, this concept might resonate with you, as it aligns with the **Decorator** design pattern. If you're not familiar, it dynamically adds behaviors to objects by wrapping them in additional objects, rather than altering their structure. This allows for greater flexibility than subclassing, as it extends functionality without modifying the original object.

Now, while HOCs remain beneficial in various scenarios for both class components and functional components, React Hooks offer a more lightweight approach to achieving composition. Let's look at Hooks next.

Exploring React Hooks

Hooks provide a means to extract stateful logic from a component, enabling its independent testing and reuse. They pave the way for reutilizing stateful logic without altering your component hierarchy. Essentially, Hooks let you “hook into” React state and other life cycle features from function components.

Following on from the `ExpandablePanel` component example, let's look at this code:

```
const useAutoClose = (duration: number) => {
  const [isOpen, setIsOpen] = useState<boolean>(true);

  useEffect(() => {
    if (isOpen) {
      const timerId = setTimeout(() => setIsOpen(false), duration);
      return () => clearTimeout(timerId);
    }
  })
}
```

```
    }, [duration, isOpen]);

    const toggle = () => setIsOpen((show) => !show);

    return { isOpen, toggle };
  };

  export default useAutoClose;
```

In this `useAutoClose` Hook, we create an `isOpen` state and a function `toggle` to switch the state. The `useEffect` function sets a timer to change `isOpen` to `false` after a specified duration, but only if `isOpen` is `true`. It also cleans up the timer to prevent memory leaks.

Now, to integrate this Hook into our `ExpandablePanel`, minimal amendments are needed:

```
const ExpandablePanel = ({ heading, content }: PanelProps) => {
  const { isOpen, toggle } = useAutoClose(2000);

  return (
    <article>
      <header onClick={toggle}>{heading}</header>
      {isOpen && <section>{content}</section>}
    </article>
  );
};
```

Here, we deleted the passed-in `isOpen` and `toggle` props and utilized the return value from the `useAutoClose` Hook, seamlessly incorporating the auto-close functionality.

Next, to incorporate keyboard navigation, we define another Hook, `useKeyboard`, which captures key events to toggle the panel:

```
const useKeyboard = (toggle: () => void) => {
  const handleKeyDown = (event: KeyboardEvent) => {
    if (event.key === "Enter" || event.key === " ") {
      event.preventDefault();
      toggle();
    }
  };

  return { handleKeyDown };
};
```

Then, embedding `useKeyboard` within `ExpandablePanel` is straightforward:

```
const ExpandablePanel = ({ heading, content }: PanelProps) => {
  const { isOpen, toggle } = useAutoClose(2000);
  const { handleKeyDown } = useKeyboard(toggle);

  return (
    <article onKeyDown={handleKeyDown} tabIndex={0}>
      <header onClick={toggle}>{heading}</header>
      {isOpen && <section>{content}</section>}
    </article>
  );
};
```

Here, `handleKeyDown` from `useKeyboard` is employed to detect key presses, enhancing our component with keyboard interactivity.

In *Figure 10.3*, you can observe how the Hooks link with the underlying `ExpandablePanel`, contrasting the HOC scenario where the component is wrapped:

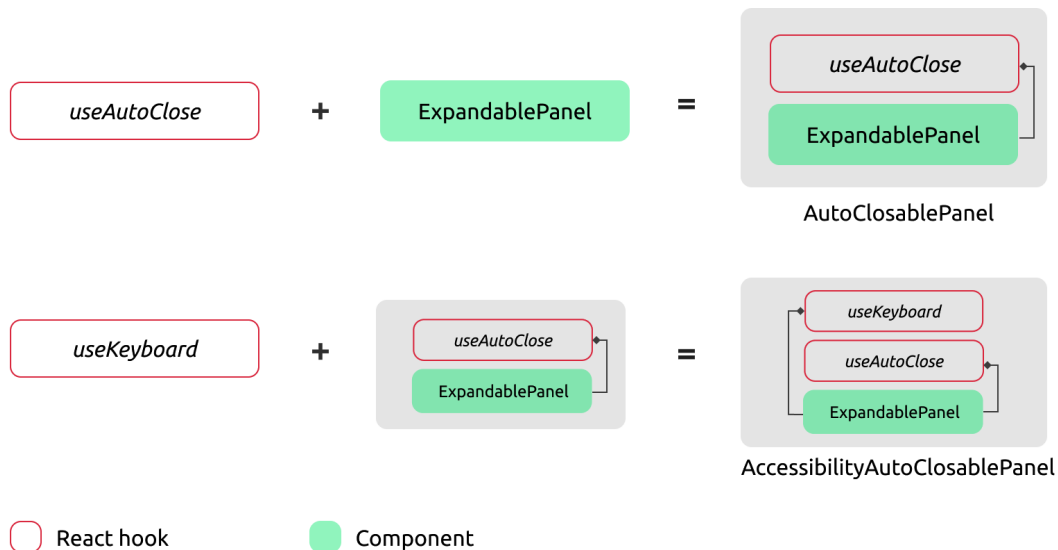


Figure 10.3: Using alternative Hooks

Hooks embody a neat package of reusable logic, isolated from the component yet easily integrated. Unlike the wrapping approach of HOCs, Hooks offer a plugin mechanism, making them lightweight and well-managed by React. This characteristic of Hooks not only promotes code modularity but also facilitates a cleaner and more intuitive way to enrich our components with additional functionalities.

However, be aware that Hooks offer more versatility than initially apparent. They're not just for managing UI-related state but are also effective for handling UI side effects, such as data fetching and global event handling (such as page-level keyboard shortcuts). We have seen how to use them for keyboard event handlers, so now, let's explore how Hooks can streamline network requests.

Unveiling remote data fetching

In previous chapters, we leveraged `useEffect` for data fetching, a prevalent approach. When retrieving data from a remote server, it typically necessitates the introduction of three distinct states – loading, error, and data.

Here's a method to implement these states:

```
//...
const [loading, setLoading] = useState<boolean>(false);
const [data, setData] = useState<Item[] | null>(null);
const [error, setError] = useState<Error | undefined>(undefined);

useEffect(() => {
  const fetchData = async () => {
    setLoading(true);

    try {
      const response = await fetch("/api/users");

      if (!response.ok) {
        const error = await response.json();
        throw new Error(`Error: ${error.error} || response.status`);
      }

      const data = await response.json();
      setData(data);
    } catch (e) {
      setError(e as Error);
    } finally {
      setLoading(false);
    }
  };

  fetchData();
}, []);

//...
```

In the preceding code, we use React Hooks to manage asynchronous data fetching, initializing states for loading, data, and error. Inside `useEffect`, the `fetchData` function attempts to retrieve user data from the `"/api/users"` endpoint. If successful, the data is stored; if not, an error is recorded. Regardless of the outcome, the loading state is updated to reflect completion. `useEffect` runs only once, similar to the component's initial mounting phase.

Refactoring for elegance and reusability

Incorporating fetching logic directly within our component can work, but it's not the most elegant or reusable approach. Let's refactor this by extracting the fetching logic into a separate function:

```
const fetchUsers = async () => {
  const response = await fetch("/api/users");

  if (!response.ok) {
    const error = await response.json();
    throw new Error('Something went wrong');
  }

  return await response.json();
};
```

With the `fetchUsers` function in place, we can take a step further by abstracting our fetching logic into a generic Hook. This Hook will accept a `fetch` function and manage the associated loading, error, and data states:

```
const useService = <T>(fetch: () => Promise<T>) => {
  const [loading, setLoading] = useState<boolean>(false);
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<Error | undefined>(undefined);

  useEffect(() => {
    const fetchData = async () => {
      setLoading(true);

      try {
        const data = await fetch();
        setData(data);
      } catch(e) {
        setError(e as Error);
      } finally {
        setLoading(false);
      }
    }
  });
};
```

```
    };

    fetchData();
  }, [fetch]);

  return {
    loading,
    error,
    data,
  };
}
```

Now, the `useService` Hook emerges as a reusable solution to fetch data across our application. It's a neat abstraction that we can employ to fetch various types of data, as seen here:

```
const { loading, error, data } = useService(fetchProducts);
//or
const { loading, error, data } = useService(fetchTickets);
```

With this refactoring, we've not only simplified our data fetching logic but also made it reusable across different scenarios in our application. This sets a solid foundation as we continue to enhance our drop-down component and delve deeper into more advanced features and optimizations.

As we have explored Hooks and their capabilities in managing state and logic, let's apply this knowledge to build a more complex UI component from scratch — a drop-down list. This exercise will not only reinforce our understanding of Hooks but also demonstrate their practical application in creating interactive UI elements.

We'll start with a basic version of a drop-down list and then gradually introduce more features to make it functional and user-friendly. This process will also set the stage for a later discussion on headless components, showcasing a design pattern that further abstracts and manages state and logic in UI components.

Developing a drop-down list component

A drop-down list is a common component used in many places. Although there's a native select component for basic use cases, a more advanced version offering more control over each option provides a better user experience.

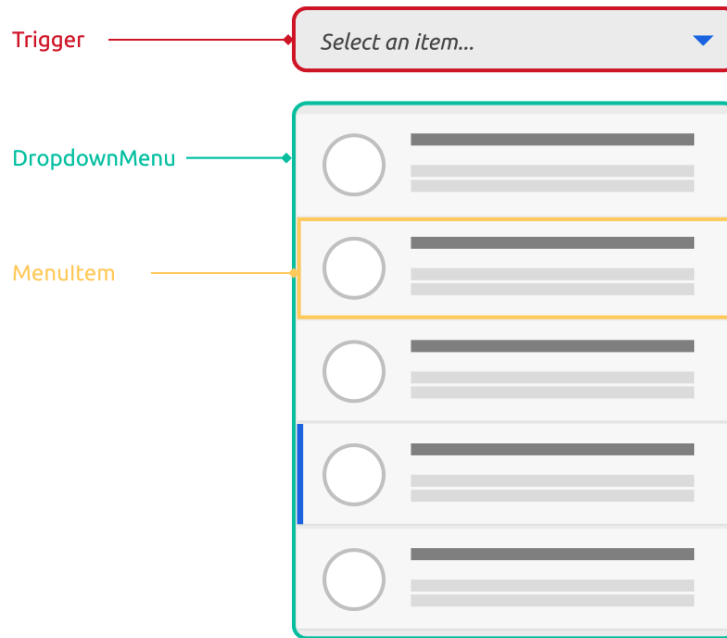


Figure 10.4: A drop-down list component

When creating one from scratch, a complete implementation requires more effort than it appears at first glance. It's essential to consider keyboard navigation, accessibility (for instance, screen reader compatibility), and usability on mobile devices, among others.

We'll begin with a simple, desktop version that only supports mouse clicks, gradually building in more features to make it realistic. Note that the goal here is to reveal a few software design patterns rather than teach you how to build a drop-down list for production use (actually, I don't recommend doing this from scratch and would instead suggest using more mature libraries).

Basically, we need an element (let's call it a trigger) for the user to click, and a state to control the show and hide actions of the list panel. Initially, we hide the panel, and when the trigger is clicked, we show the list panel. Here is the code:

```
import { useState } from "react";

interface Item {
  icon: string;
  text: string;
  id: string;
  description: string;
```



```

}

type DropdownProps = {
  items: Item[];
};

const Dropdown = ({ items }: DropdownProps) => {
  const [isOpen, setIsOpen] = useState(false);
  const [selectedItem, setSelectedItem] = useState<Item | null>(null);

  return (
    <div className="dropdown">
      <div className="trigger" tabIndex={0} onClick={() =>
        setIsOpen(!isOpen)}>
        <span className="selection">
          {selectedItem ? selectedItem.text : "Select an item..."}
        </span>
      </div>
      {isOpen && (
        <div className="dropdown-menu">
          {items.map((item) => (
            <div
              key={item.id}
              onClick={() => setSelectedItem(item)}
              className="item-container"
            >
              <img src={item.icon} alt={item.text} />
              <div className="details">
                <div>{item.text}</div>
                <small>{item.description}</small>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
};

```

In the code, we've set up the basic structure for our drop-down component. Using the `useState` Hook, we manage the `isOpen` and `selectedItem` states to control the dropdown's behavior. A simple click on the trigger toggles the drop-down menu, while selecting an item updates the `selectedItem` state.

Let's break down the component into smaller, manageable pieces to see it more clearly. We'll start by extracting a Trigger component to handle user clicks:

```
const Trigger = ({
  label,
  onClick,
}): {
  label: string;
  onClick: () => void;
} => {
  return (
    <div className="trigger" tabIndex={0} onClick={onClick}>
      <span className="selection">{label}</span>
    </div>
  );
};
```

Similarly, we'll extract a DropdownMenu component to render the list of items:

```
const DropdownMenu = ({
  items,
  onItemClick,
}): {
  items: Item[];
  onItemClick: (item: Item) => void;
} => {
  return (
    <div className="dropdown-menu">
      {items.map((item) => (
        <div
          key={item.id}
          onClick={() => onItemClick(item)}
          className="item-container"
        >
          <img src={item.icon} alt={item.text} />
          <div className="details">
            <div>{item.text}</div>
            <small>{item.description}</small>
          </div>
        </div>
      ))}
    </div>
  );
};
```

Now, in the Dropdown component, we simply use these two components, passing in the corresponding state, turning them into purely controlled components (stateless components):

```
const Dropdown = ({ items }: DropdownProps) => {
  const [isOpen, setIsOpen] = useState(false);
  const [selectedItem, setSelectedItem] = useState<Item | null>(null);

  return (
    <div className="dropdown">
      <Trigger
        label={selectedItem ? selectedItem.text : "Select an item..."}
        onClick={() => setIsOpen(!isOpen)}
      />
      {isOpen && <DropdownMenu items={items}
        onItemClick={setSelectedItem} />}
    </div>
  );
};
```

In this updated code structure, we've separated concerns by creating specialized components for different parts of the dropdown, making the code more organized and easier to manage. We can see the result here:

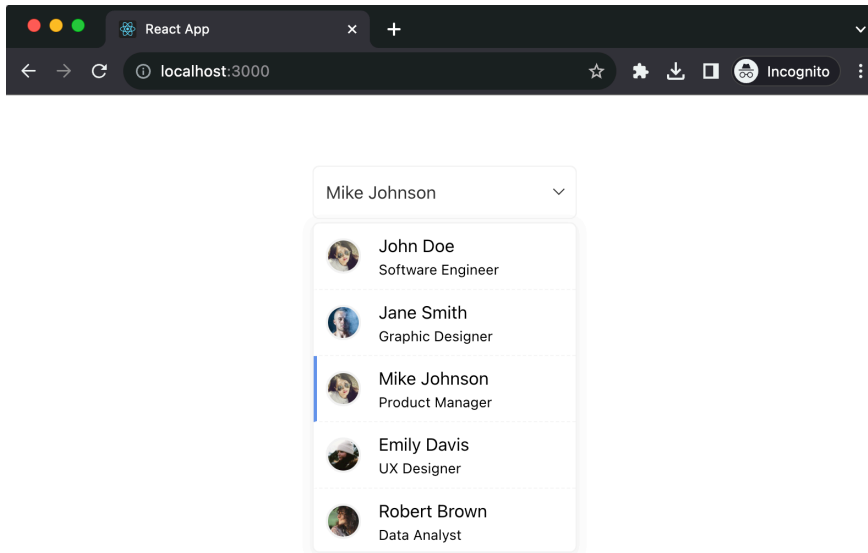


Figure 10.5: A native implementation list

As you can see in *Figure 10.5*, the basic drop-down list appears, but that's only a small part of the whole drop-down list features. Keyboard navigation, for example, is a necessary feature for an accessible component, which we will implement next.

Implementing keyboard navigation

Incorporating keyboard navigation within our drop-down list enhances the user experience by providing an alternative to mouse interactions. This is particularly important for accessibility and offers a seamless navigation experience on a web page. Let's explore how we can achieve this using the `onKeyDown` event handler.

Initially, we'll attach a `handleKeyDown` function to the `onKeyDown` event in our `Dropdown` component. Here, we utilize a `switch` statement to determine the specific key pressed and perform actions accordingly. For instance, when the `Enter` or `Space` key is pressed, the dropdown is toggled. Similarly, the `ArrowDown` and `ArrowUp` keys allow navigation through the list items, cycling back to the start or end of the list when necessary:

```
const Dropdown = ({ items }: DropdownProps) => {
  // ... previous state variables ...

  const handleKeyDown = (e: React.KeyboardEvent) => {
    switch (e.key) {
      // ... case blocks ...
    }
  };

  return (
    <div className="dropdown" onKeyDown={handleKeyDown}>
      {/* ... rest of the JSX ... */}
    </div>
  );
};
```

Additionally, we have updated our `DropdownMenu` component to accept a `selectedIndex` prop. This prop is used to apply a highlighted style and set the `aria-selected` attribute to the currently selected item, enhancing the visual feedback and accessibility:

```
const DropdownMenu = ({
  items,
  selectedIndex,
  onItemClick,
}: {
  items: Item[];
  selectedIndex: number;
  onItemClick: (item: Item) => void;
}) => {
  return (
    <div className="dropdown-menu" role="listbox">
```

```
    { /* ... rest of the JSX ... */ }  
  </div>  
);  
};
```

Moving forward, we can encapsulate the state and keyboard event handling logic within a custom Hook named `useDropdown`. This Hook returns an object containing the necessary states and functions, which can be destructured and used within our `Dropdown` component, keeping it clean and maintainable:

```
const useDropdown = (items: Item[]) => {  
  // ... state variables ...  
  
  const handleKeyDown = (e: React.KeyboardEvent) => {  
    // ... switch statement ...  
  };  
  
  const toggleDropdown = () => setIsOpen((isOpen) => !isOpen);  
  
  return {  
    isOpen,  
    toggleDropdown,  
    handleKeyDown,  
    selectedItem,  
    setSelectedItem,  
    selectedIndex,  
  };  
};
```

Now, our `Dropdown` component is simplified and more readable; it leverages the `useDropdown` Hook to manage its state and handle keyboard interactions, demonstrating a clear separation of concerns and making the code easier to understand and manage:

```
const Dropdown = ({ items }: DropdownProps) => {  
  const {  
    isOpen,  
    selectedItem,  
    selectedIndex,  
    toggleDropdown,  
    handleKeyDown,  
    setSelectedItem,  
  } = useDropdown(items);  
  
  return (  

```

```

<div className="dropdown" onKeyDown={handleKeyDown}>
  <Trigger
    onClick={toggleDropdown}
    label={selectedItem ? selectedItem.text : "Select an item..."}
  />
  {isOpen && (
    <DropdownMenu
      items={items}
      onItemClick={setSelectedItem}
      selectedIndex={selectedIndex}
    />
  )}
</div>
);
};

```

Through these modifications, we have successfully implemented keyboard navigation in our drop-down list, making it more accessible and user-friendly. This example also illustrates how Hooks can be utilized to manage complex state and logic in a structured and modular manner, paving the way for further enhancements and feature additions to our UI components.

We can visualize the code a bit better with the React DevTools. Note that in the **hooks** section, all the states are listed:

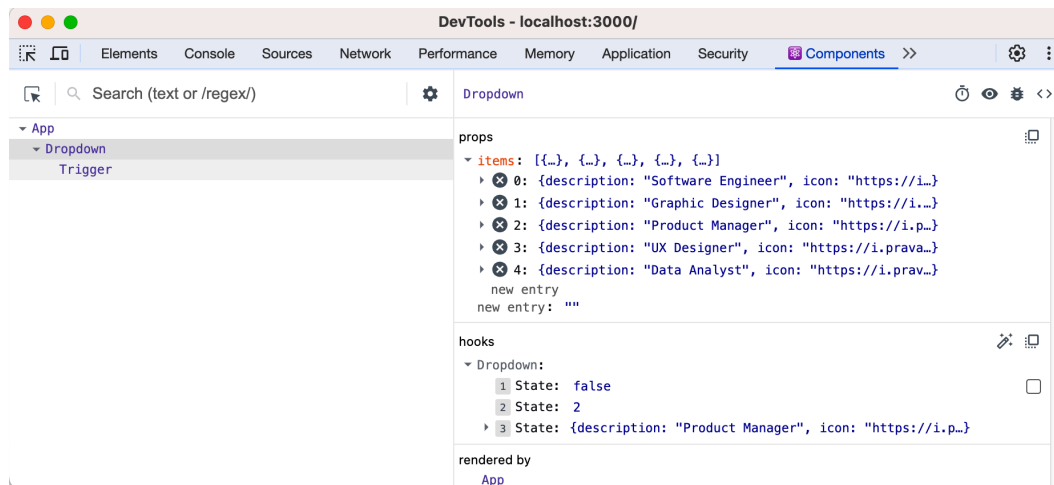


Figure 10.6: Chrome DevTools to inspect what's in the hooks section

The power of extracting our logic into a Hook comes into full play when we need to implement a different UI while maintaining the same underlying functionality. By doing so, we've segregated our state management and interaction logic from the UI rendering, making it a breeze to change the UI without touching the logic.

We've explored how utilizing small components with custom Hooks can enhance our code structure. However, what happens when we're faced with managing more complex states? Consider a scenario where the drop-down data comes from a service API, requiring us to handle asynchronous service calls along with additional state management. In such cases, does this structure still hold up effectively?

The scenario of fetching data from a remote source brings forth the necessity to manage a few more states – specifically, we need to handle loading, error, and data states. As illustrated in *Figure 10.7*, besides displaying a regular list, we also aim to manage scenarios where the data is not immediately accessible – either when it's still loading from a remote API or it isn't available.

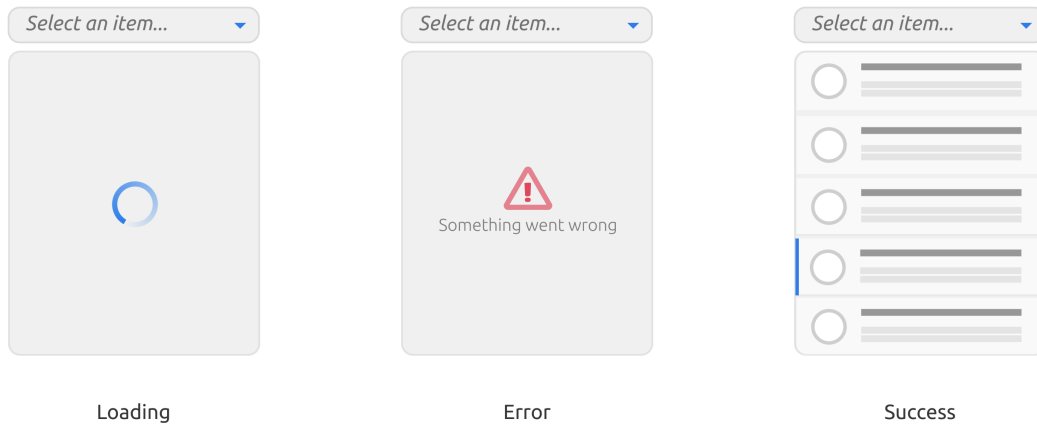


Figure 10.7: Different statuses

Such states, while common, are crucial for the user experience. Take, for instance, a drop-down list featuring country names. It's a common feature, yet when we open the list, the names might still be loading, during which a loading indicator is displayed. Additionally, in situations where the downstream service is unavailable or other errors arise, an error message is presented instead.

When extending our existing code, it's crucial to deliberate on the additional states that will be introduced. Let's explore strategies to preserve simplicity as we integrate new features.

Maintaining simplicity in the Dropdown component

Incorporating remote data fetching has not complicated our Dropdown component, thanks to the abstracted logic in the `useService` and `useDropdown` Hooks. Our component code remains in its simplest form, effectively managing the fetching states and rendering the content based on the data received:

```
const Dropdown = () => {  
  const { data, loading, error } = useService(fetchUsers);  
  
  const {
```

```

    toggleDropdown,
    dropdownRef,
    isOpen,
    selectedItem,
    selectedIndex,
    updateSelectedItem,
    getAriaAttributes,
  } = useDropdown<Item>(data || []);

const renderContent = useCallback(() => {
  if (loading) return <Loading />;
  if (error) return <Error />;
  if (data) {
    return (
      <DropdownMenu
        items={data}
        updateSelectedItem={updateSelectedItem}
        selectedIndex={selectedIndex}
      />
    );
  }
  return null;
}, [loading, error, data, updateSelectedItem, selectedIndex]);

return (
  <div
    className="dropdown"
    ref={dropdownRef as RefObject<HTMLDivElement>}
    {...getAriaAttributes()}
  >
    <Trigger
      onClick={toggleDropdown}
      text={selectedItem ? selectedItem.text : "Select an item..."}
    />
    {isOpen && renderContent()}
  </div>
);
};

```

In this updated Dropdown component, we utilize the `useService` Hook to manage the data fetching states and the `useDropdown` Hook to manage the drop-down-specific states and interactions. The `renderContent` function elegantly handles the rendering logic based on the fetching states, ensuring that the correct content is displayed, whether it's loading, an error, or data.

Through the separation of concerns and the use of Hooks, our Dropdown component remains clean and straightforward, showcasing the power of composable logic in React. Now, this pattern actually has a particular name in building UI – the Headless Component pattern. Let’s look at it in more detail.

Introducing the Headless Component pattern

The **Headless Component pattern** unveils a robust avenue to cleanly segregate our JSX code from the underlying logic. While composing a declarative UI with JSX comes naturally, the real challenge lies in managing state. This is where headless components come into play, by shouldering all the state management intricacies and propelling us toward a new horizon of abstraction.

In essence, a **Headless Component** is a function or object that encapsulates logic but doesn’t render anything itself. It leaves the rendering part to the consumer, thus offering a high degree of flexibility in how the UI is rendered. This pattern can be exceedingly useful when we have complex logic that we want to reuse across different visual representations.

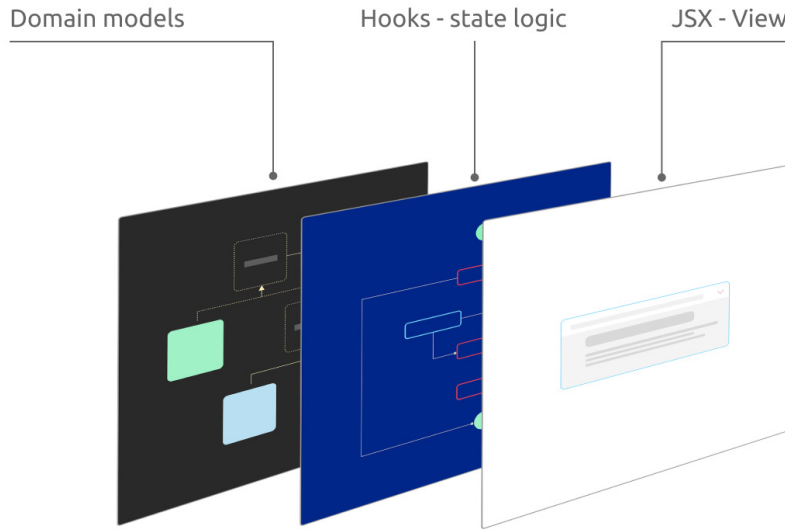
As shown in the following code, the `useDropdownLogic` Hook has all the logic but no UI elements, while `MyDropdown` uses the headless component and only has to deal with rendering logic:

```
function useDropdownLogic() {
  // ... all the dropdown logic
  return {
    // ... exposed logic
  };
}

function MyDropdown() {
  const dropdownLogic = useDropdownLogic();
  return (
    // ... render the UI using the logic from dropdownLogic
  );
}
```

In a visual representation, the headless component acts as a thin interface layer. On one side, it interacts with JSX views, and on the other, it communicates with underlying data models. We touched upon data modeling in *Chapter 8* and will revisit it in *Chapter 11*. This pattern is particularly beneficial for individuals seeking solely the behavior or state management aspect of the UI, as it conveniently segregates it from the visual representation.

Let’s see a visual illustration in *Figure 10.8*. You can think of your code as having a few layers – the JSX is on the top and is response for the look and feel part of the application, the headless component (Hooks in this case) manages all the stateful logic, and beneath them is the domain layer, which has the logic to handle data mapping and transformation (we will go into more detail on this in *Chapter 11* and *Chapter 12*).



Using hooks as the headless component

Figure 10.8: The Headless Component pattern

In summarizing the Headless Component pattern, it's worth mentioning that although it can be realized through HOCs or render props, its implementation as a React Hook is more prevalent. Within the Headless Component pattern, all shareable logic is encapsulated, allowing for a seamless transition to other UIs without necessitating any modifications to the stateful logic.

The advantages and drawbacks of Headless Component pattern

The advantages of the Headless Component pattern include the following:

- **Reusability:** The logic encapsulated in Headless Component pattern can be reused across multiple components. This fosters **Don't Repeat Yourself (DRY)** principles in your codebase.
- **Separation of concerns:** By decoupling logic from rendering, headless components promote a clear separation of concerns, which is a cornerstone of maintainable code.
- **Flexibility:** They allow for varying UI implementations while sharing the same core logic, making it easier to adapt to different design requirements or frameworks.

The drawbacks of Headless Component pattern include the following:

- **Learning curve:** The pattern may introduce a learning curve for developers unfamiliar with it, potentially slowing down development initially.
- **Over-abstraction:** If not managed judiciously, the abstraction created by headless components can lead to a level of indirection that might make the code harder to follow.

Libraries and further learnings

The Headless Component pattern has been embraced by various libraries to facilitate the creation of accessible, customizable, and reusable components. Here are some notable libraries along with a brief description of each:

- **React Aria:** A library from Adobe that provides accessibility primitives and Hooks to build inclusive React applications. It offers a collection of Hooks to manage keyboard interactions, focus management, and Aria annotations, making it easier to create accessible UI components.
- **Headless UI:** A completely unstyled, fully accessible UI component library, designed to integrate beautifully with Tailwind CSS. It provides the behavior and accessibility foundation upon which you can build your own styled components.
- **React Table:** A headless utility for building fast and extendable tables and data grids for React. It provides a flexible Hook that allows you to create complex tables with ease, leaving the UI representation up to you.
- **Downshift:** A minimalist library to help you create accessible and customizable dropdowns, comboboxes, and so on. It handles all the logic while letting you define the rendering aspect.

These libraries embody the essence of the Headless Component pattern by encapsulating complex logic and behaviors, making it straightforward to create highly interactive and accessible UI components. While the provided example serves as a learning stepping stone, it's prudent to leverage these production-ready libraries to build robust, accessible, and customizable components in a real-world scenario.

This pattern not only educates us on managing complex logic and state but also encourages us to explore production-ready libraries that have honed the Headless Component approach, delivering robust, accessible, and customizable components for real-world use.

Summary

In this chapter, we delved into the world of HOCs and Hooks in React, exploring their utility in enhancing component logic while maintaining a clean, readable code base. Through the lens of creating an expandable panel and a drop-down list, we illustrated the composability of HOCs and the encapsulation of stateful logic that Hooks offer. Transitioning to a more intricate drop-down list, we introduced asynchronous data fetching, demonstrating how Hooks can simplify state management in data-loading scenarios.

We then transitioned into the realm of Headless Component, a powerful pattern that separates logic from the JSX code, providing a robust framework to manage state while leaving the UI representation to the developer. Through examples, we demonstrated how this separation facilitates the creation of reusable, accessible, and customizable components. The discussion was enriched with a review of notable libraries, such as React Table, Downshift, React Aria, and Headless UI, that embody the Headless Component pattern, providing ready-to-use solutions to build interactive and accessible UI components.

In the upcoming chapter, we'll implement the patterns we've discussed and delve into architectural strategies to enhance modularity. We'll also address the challenges posed by larger applications.

Part 4:

Engaging in Practical Implementation

In the final part of this book, you will apply your accumulated knowledge in a hands-on manner by employing a layered architecture in React and journeying through an end-to-end project implementation. This part aims to encapsulate the essence of all the principles, patterns, and practices discussed throughout the book.

This part contains the following chapters:

- *Chapter 11, Introducing Layered Architecture in React*
- *Chapter 12, Implementing an End-To-End Project*
- *Chapter 13, Recapping Anti-Pattern Principles*

Introducing Layered Architecture in React

As React applications grow in size and complexity, managing code efficiently becomes a challenge. The linear growth of features can lead to an exponential increase in complexity, making the code base difficult to understand, test, and maintain. Enter **Layered Architecture**, a design approach that's not just confined to backend systems but is equally beneficial for client-side applications.

Structuring your React application in a layered manner solves several key problems:

- **Separation of concerns:** Different layers handle different responsibilities, making the code base easier to navigate and understand
- **Reusability:** Business logic and data models become easily reusable across different parts of the application
- **Testability:** A layered architecture makes it simpler to write unit and integration tests, leading to a more robust application
- **Maintainability:** As the application scales, making changes or adding features becomes significantly easier when following a layered structure

In this chapter, we will explore the concept of layered architecture in the context of a React application, delving into the extraction of application concern layers, defining precise data models, and illustrating the use of strategy patterns. Through a step-by-step example, we'll see how to practically implement these concepts and why they are indispensable for large-scale applications.

In this chapter, we will cover the following topics:

- Understanding the evolution of a React application
- Enhancing the Code Oven application
- Implementing the ShoppingCart component
- Delving into layered architecture

Technical requirements

A GitHub repository has been created to host all the code we discuss in the book. For this chapter, you can find the recommended structure at <https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch11>.

Understanding the evolution of a React application

Applications with different sizes require different strategies. For small or one-off projects, you might find that all logic is just written inside React components. You may see one or only a few components in total. The code looks pretty much like HTML, with only some variables or states used to make the page “dynamic,” but overall, the code is easy to understand and change.

As the application grows, and more and more code is added to the code base, without a proper way to organize it, the code base will soon get into an unmaintainable state. This means that even adding small features will be time-consuming as developers need more time to read the code.

In this section, I’ll list several different ways we can structure our React application to make sure our code always remains in a healthy state, making it effortless to add new features and easy to extend or fix existing defects. We’ll start with a simple structure and gradually evolve it to handle scale problems. Let’s have a quick review of the steps to build frontend applications that scale.

Single-component applications

To start, let’s talk about the simplest possible approach to writing a React application – a single-component application.

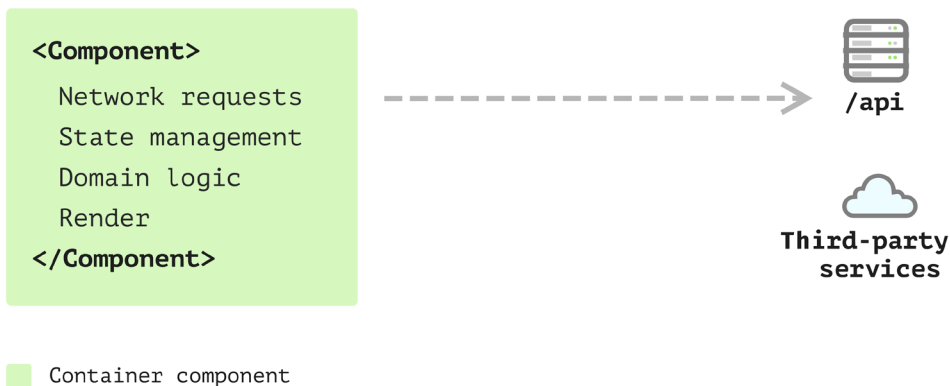


Figure 11.1: Single-component application

The mono-component undertakes a variety of tasks, ranging from fetching data from a remote server, managing its internal state, and handling domain logic, to rendering. This approach may be suitable for small applications with a single form or for those looking to understand the process of transitioning their application from another framework to React.

However, you'll soon realize that consolidating everything into a single component can make the code difficult to understand and manage. Everything being housed in one component can quickly become overwhelming, particularly when dealing with logic such as iterating over item lists to create individual components. This complexity highlights the need to decompose the mono-component into smaller, responsibility-focused components.

Multiple-component applications

Deciding to split the component into several components, with these structures reflecting what's happening on the resulting HTML, is a good idea and it helps you to focus on one component at a time.

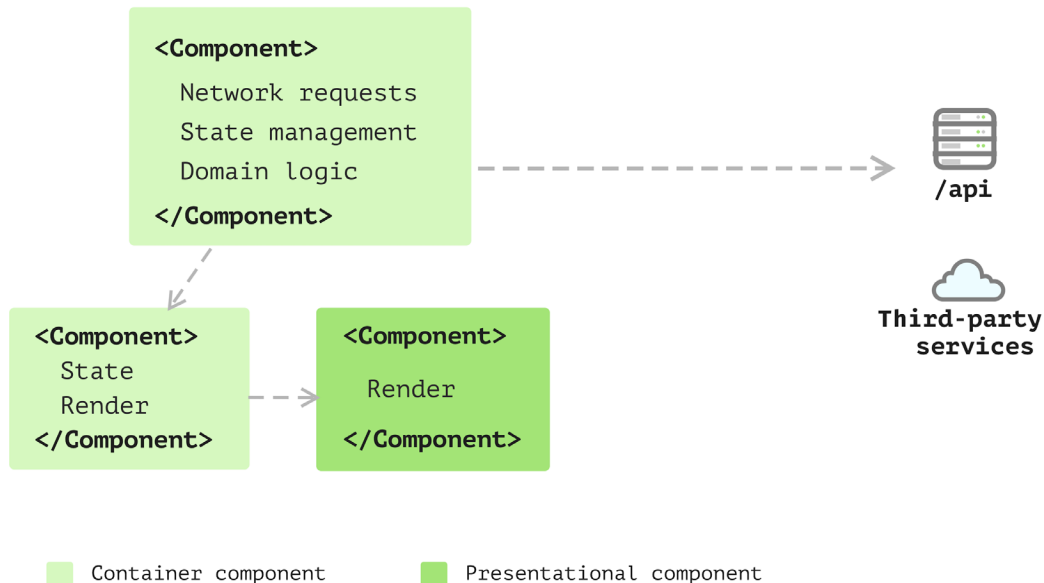


Figure 11.2: Multiple-component application

Essentially, you'll transition from a monolithic component to multiple components, each with a specific purpose. For example, one component may be dedicated to rendering a list, another to rendering a list item, and another solely for fetching data and passing data down to its children.

It's better to have clear responsibilities. However, as your application expands, responsibilities extend beyond the view layer to include tasks such as sending network requests, reshaping data for the view to consume, and collecting data to send back to the server. Additionally, there might be logic to transform data once it's fetched. Housing this calculation logic within views doesn't seem appropriate as it's not directly related to user interfaces. Moreover, some components may become cluttered with excessive internal states.

State management with Hooks

It's a better idea to split this logic into separate places. Luckily, in React, you can define your own Hooks. This is a great way to share this state and the logic whenever state changes.

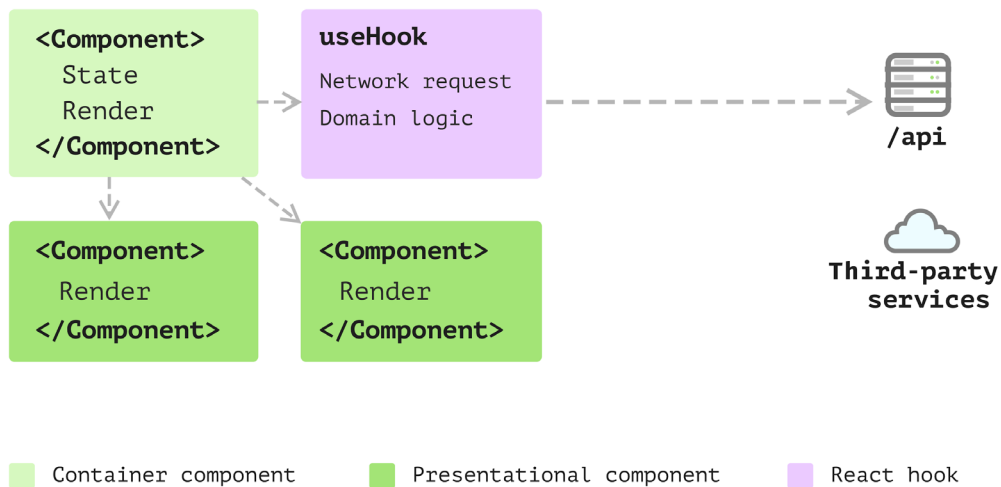


Figure 11.3: State management with Hooks

Now you have a bunch of elements extracted from components. You have a few pure presentational components, some reusable Hooks that make other components stateful, and some container components (for data fetching, for example).

At this stage, it's common to find calculations scattered across views, Hooks, or various utility functions. The lack of structure can make further modifications quite challenging and prone to errors. For instance, if you've fetched some data for rendering, but the data schema differs in views, you'll need to transform the data. However, the location for placing this transforming logic may not be clear.

Extracting business models

So, you've started to become aware that extracting this logic into yet another place can bring you many benefits. For example, with that split, the logic can be cohesive and independent of any views. Then, you extract a few domain objects.

These simple objects can handle data mapping (from one format to another), check nulls, and use fallback values as required. As the amount of these domain objects grows, you will find you need some inheritance or polymorphism to make things even cleaner. Thus you apply many design patterns you find helpful from other places to the frontend application:

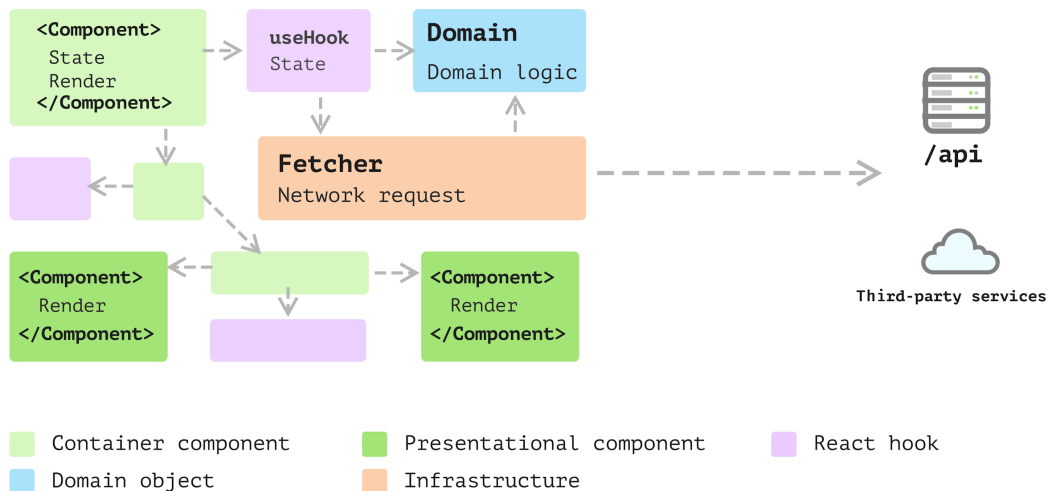


Figure 11.4: Extracting business models

Now, your code base has expanded with more elements, each having a clear boundary regarding their responsibilities. Hooks are employed for state management, while domain objects represent domain concepts, such as a user object encompassing an avatar, or a `PaymentMethod` object representing the details of a payment method.

As we segregate different elements from the views, the code base scales accordingly. Eventually, it reaches a point where we need to structure the application to respond to changes more efficiently.

Layered frontend application

As the application continues to evolve, certain patterns begin to emerge. You'll notice a collection of objects that don't belong to any user interface, and they remain indifferent to whether the underlying data originates from a remote service, local storage, or cache. Consequently, you'll want to segregate them into distinct layers. We'll need to introduce a better approach for these different parts of the application.